

• SENACGAMES • CURSO TÉCNICO EM INFORMÁTICA

CRUD DE USUÁRIOS

MANUAL DE IMPLEMENTAÇÃO PASSO A PASSO

Ensinar, passo a passo, como desenvolver o módulo completo de Usuários utilizando a arquitetura em camadas já existente da solução SenacGames, integrando com o ASP.NET Core Identity.

DTOs

ASP.NET Identity

REST API

Injeção de Dependência

Windows Forms

Guna UI2

VISÃO GERAL DO MANUAL

OBJETIVO

Ensinar, passo a passo, como desenvolver o módulo completo de Usuários utilizando a arquitetura em camadas já existente da solução SenacGames, integrando com o ASP.NET Core Identity.

Pré-requisitos

Conhecimentos básicos de C# e Orientação a Objetos.
Compreensão básica sobre requisições HTTP (GET, POST, PUT, DELETE).
Familiaridade com o Visual Studio 2022.

Tempo Estimado

3 a 4 horas.

COMPETÊNCIAS DESENVOLVIDAS

- Compreensão prática de Arquitetura em Camadas (Domain, Application, API, Desktop).
- Criação e uso de DTOs (Data Transfer Objects).
- Injeção de Dependência no ASP.NET Core.
- Criação de APIs RESTful usando Controllers.
- Integração e uso do ASP.NET Core Identity (userManager, RoleManager).
- Consumo de API em aplicações Windows Forms (Desktop) utilizando HttpClient.
- Construção de interfaces ricas com Guna UI2 (DataGridView, Forms).

SUMÁRIO

MANUAL

- 01** A Arquitetura da Solução
- 02** Analisando Módulos Existentes
- 03** Planejamento do CRUD de Usuários
- 04** Implementação Passo a Passo

- 05** Debugging e Resolução de Problemas

PARA PRATICAR

- 06** Exercícios de Fixação
- 07** Desafios Extras
- 08** Checklist de Sucesso

A ARQUITETURA DA SOLUÇÃO

O SenacGames adota a **Arquitetura em Camadas** (N-Tier Architecture), cujo objetivo é separar as responsabilidades do sistema. Cada projeto na solução tem uma função específica, e eles "conversam" entre si de maneira muito organizada.

- **SenacGames.Domain** : O "coração" do sistema. Contém as regras de negócio puras, entidades (classes que representam as tabelas do banco) e as interfaces dos repositórios.
- **SenacGames.Infrastructure** : A ponte com o mundo externo. É aqui que o Entity Framework Core "mora" e onde o acesso ao banco de dados acontece.
- **SenacGames.Application** : A camada "maestro". Ela recebe os pedidos da API, traduz os dados usando **DTOs**, aplica a lógica e delega as operações de banco para a camada de Infraestrutura.
- **SenacGames.API** : A porta de entrada da web. Contém os **Controllers**, que recebem as requisições HTTP (via Front-end, Mobile ou Desktop) e repassam para a camada Application.
- **SenacGames.Desktop** (UI): O cliente. É o aplicativo Windows Forms que o usuário final utiliza. Ele não se conecta ao banco de dados, mas sim consome a API através de serviços HTTP.

Fluxo de Comunicação Textual

```
[Desktop] (Tela do Usuário)
  ↓ (Chama um ApiService)
[API] (Recebe a requisição HTTP no Controller)
  ↓ (Chama um Service)
[Application] (Processa regras, mapeia DTOs)
  ↓ (Chama Repositório / Identity)
[Infrastructure] (Entity Framework Core)
  ↓
[Banco de Dados] (SQL Server)
```

02 MANUAL ANALISANDO MÓDULOS EXISTENTES

Antes de criar um módulo novo, é fundamental estudar como os módulos já existentes (como **Games** e **Categorias**) foram construídos. Assim, nós mantemos o **padrão** do sistema.

Se você observar o módulo de Categorias:

- 1 **Application/DTOs:** existe um `CategoryDto` (para mostrar os dados) e um `CreateCategoryDto` (para criar).
- 2 **Application/Services:** o `CategoryService` contém os métodos `GetAllAsync`, `CreateAsync`, etc. Ele recebe os DTOs e retorna os resultados.
- 3 **API/Controllers:** o `CategoriesController` expõe rotas como `[HttpGet] /api/categories`.
- 4 **Desktop/Services:** o `CategoriasApiService` faz um `HttpClient.GetAsync` e desserializa o JSON.
- 5 **Desktop/UserControls:** o `CategoriasUserControl` usa um `DataGridView` para mostrar tudo em tela.

No CRUD de Usuários, faremos **exatamente** o mesmo fluxo!

Vamos planejar nossa implementação. Precisamos criar os seguintes arquivos:

Na Camada Application (Regra de Negócio)

- `UsuarioDto.cs` : para transferir os dados sem expor as senhas diretamente.
- `IUsuariosService.cs` : o contrato contendo o que nosso serviço sabe fazer.
- `UsuariosService.cs` : a implementação real, que vai usar o Identity para gravar os usuários.

Na Camada API (Exposição REST)

- `UsuariosController.cs` : os endpoints REST que o Desktop vai consumir.

Na Camada Desktop (Tela do Usuário)

- `UsuarioDtos.cs` : espelho dos DTOs da API, para desserializar o JSON.
- `UsuariosApiService.cs` : para fazer o *fetch* (GET, POST) lá na API.
- `UsuarioFormDialog.cs` : a janelinha pop-up onde o admin vai digitar o nome, email e senha.
- `UsuariosUserController.cs` : a tela principal com a grade (grid) de usuários.

04 MANUAL IMPLEMENTAÇÃO PASSO A PASSO

1. DTOs (Data Transfer Objects)

Objetivo: criar as classes que representam os dados transitando entre a API e as demais camadas.

Por quê? Evita o envio de entidades pesadas ou dados sensíveis, garantindo segurança e melhor performance.

Crie o arquivo `UsuarioDto.cs` na pasta `DTOs` no projeto **SenacGames.Application**:

```
namespace SenacGames.Application.DTOs
{
    // DTO usado para listar (repare que não enviamos a senha para a tela!)
    public class UsuarioDto
    {
        public string Id { get; set; } = string.Empty;
        public string Nome { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Perfil { get; set; } = string.Empty;
    }

    // DTO usado APENAS quando formos criar um usuário novo
    public class CreateUsuarioDto
    {
        public string Nome { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Senha { get; set; } = string.Empty;
        public string ConfirmarSenha { get; set; } = string.Empty;
        public string Perfil { get; set; } = string.Empty;
    }

    // DTO para atualizar (senha é opcional)
    public class UpdateUsuarioDto
    {
        public string Nome { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string? Senha { get; set; }
        public string? ConfirmarSenha { get; set; }
        public string Perfil { get; set; } = string.Empty;
    }
}
```

2. Interface do Serviço (IUsuariosService)

Na pasta `Interfaces` de `SenacGames.Application`, crie o contrato:

```
using SenacGames.Application.DTOs;

namespace SenacGames.Application.Interfaces
```

3. Implementação do Serviço (UsuariosService) com Identity

Por que Identity? O ASP.NET Core Identity já fornece uma infraestrutura robusta para senhas criptografadas (hash), roles (perfis) e autenticação. Não precisamos reinventar a roda criando uma tabela manual de usuários. Usaremos o `UserManager` e o `RoleManager`.

```
using Microsoft.AspNetCore.Identity;
using SenacGames.Application.DTOS;
using SenacGames.Application.Interfaces;

namespace SenacGames.Application.Services
{
    public class UsuariosService : IUserariosService
    {
        // O ASP.NET Injeta (Dependency Injection) essas classes automaticamente pra
        nós!

        private readonly UserManager<IdentityUser> _userManager;
        private readonly RoleManager<IdentityRole> _roleManager;

        public UsuariosService(UserManager<IdentityUser> userManager,
            RoleManager<IdentityRole> roleManager)
        {
            _userManager = userManager;
            _roleManager = roleManager;
        }

        public async Task<IEnumerable<UsuarioDto>> GetAllAsync()
        {
            var users = _userManager.Users.ToList();
            var result = new List<UsuarioDto>();

            // Iteramos sobre os usuários do banco e transformamos em UsuarioDto
            foreach (var user in users)
            {
                var roles = await _userManager.GetRolesAsync(user);
                result.Add(new UsuarioDto
                {
                    Id = user.Id,
                    Nome = user.UserName ?? string.Empty,
                    Email = user.Email ?? string.Empty,
                });
            }
        }
    }
}
```



```

        Perfil = roles.FirstOrDefault() ?? "Usuario"
    });
}
return result;
}

public async Task<(bool Success, UsuarioDto? Usuario, string ErrorMessage)>
CreateAsync(CreateUsuarioDto dto)
{
    // Validação simples
    if (dto.Senha != dto.ConfirmarSenha)
        return (false, null, "As senhas não coincidem.");

    // Criar o modelo base do Identity
    var user = new IdentityUser { UserName = dto.Nome, Email = dto.Email };

    // Aqui a mágica do Hash de senha acontece
    var result = await _userManager.CreateAsync(user, dto.Senha);

    if (!result.Succeeded) return (false, null, "Erro ao criar usuário.");

    // Adiciona o perfil (ex: "Admin" ou "Usuario")
    await _userManager.AddToRoleAsync(user, dto.Perfil);

    var createdUser = new UsuarioDto { Id = user.Id, Nome = user.UserName,
    Email = user.Email, Perfil = dto.Perfil };
    return (true, createdUser, string.Empty);
}

// ... (Para encurtar, implemente Update, Delete e GetById de maneira similar)
}
}

```



DICA

Note que usamos Tuplas `(bool Success, UsuarioDto? Usuario, string ErrorMessage)` como retorno. É uma prática muito boa do C# para retornar status da operação sem lançar *Exceptions* caríssimas a todo instante.

4. Controller (UsuariosController) na API

Crie `UsuariosController.cs` dentro da pasta `Controllers` no **SenacGames.API**:

```

using Microsoft.AspNetCore.Authorization;
using Microsoft.AspNetCore.Mvc;
using SenacGames.Application.DTOs;
using SenacGames.Application.Interfaces;

namespace SenacGames.API.Controllers
{
    [ApiController]           // Define que esta classe responde a requisições HTTP
    (JSON)
    [Route("api/[controller]")] // A rota será: localhost:porta/api/usuarios
    [Authorize]               // Exige que o cliente esteja logado com um token/cookie
    public class UsuariosController : ControllerBase
    {

```

```

private readonly IUserariosService _usuariosService;

public UsuariosController(IUserariosService usuariosService)
{
    _usuariosService = usuariosService;
}

[HttpGet] // GET /api/usuarios
public async Task<ActionResult<IEnumerable<UsuarioDto>>> GetAll()
{
    var usuarios = await _usuariosService.GetAllAsync();
    return Ok(usuarios); // Retorna HTTP 200 com a lista em JSON
}

[HttpPost] // POST /api/usuarios
public async Task<ActionResult<UsuarioDto>> Create([FromBody] CreateUsuarioDto
dto)
{
    var (success, usuario, error) = await _usuariosService.CreateAsync(dto);
    if (!success) return BadRequest(new { message = error }); // HTTP 400

    return Ok(usuario); // HTTP 200
}
}
}

```



ATENÇÃO (INJEÇÃO DE DEPENDÊNCIA)

Após criar a interface e o serviço, você DEVE ir no arquivo `Program.cs` da API e avisar ao ASP.NET Core que eles existem:

```
builder.Services.AddScoped<IUserariosService, UsuariosService>();
```

5. Testes da API (Swagger)

Rode a API apertando `F5`. O **Swagger** abrirá. O Swagger é uma documentação viva da sua API.

Como colocamos `[Authorize]`, primeiro você precisa usar o endpoint `POST /api/auth/login` para autenticar. Em seguida, procure a seção de `Usuarios`, clique em **Try it out** no método `GET`, e receba um JSON contendo seus usuários!

6. Consumindo a API no Desktop (UsuariosApiService)

No projeto **SenacGames.Desktop**, pasta `Services`, crie o serviço que converte os botões do formulário em chamadas de rede.

```

using SenacGames.Desktop.DTOS;
using SenacGames.Desktop.Helpers;

namespace SenacGames.Desktop.Services
{
    public class UsuariosApiService
    {
        // O HttpClientHelper faz o trabalho pesado de converter classes para JSON
        private readonly HttpClientHelper _http;
    }
}

```

```

public UsuariosApiService()
{
    _http = HttpClientHelper.Instance;
}

public async Task<List<UsuarioResponseDto>> GetAllAsync()
{
    try
    {
        // Faz a chamada na URL e transforma o JSON retornado numa List C#
        var (success, usuarios, errorMessage) = await
        _http.GetAsync<List<UsuarioResponseDto>>("/api/usuarios");
        return usuarios ?? new List<UsuarioResponseDto>();
    }
    catch
    {
        return new List<UsuarioResponseDto>();
    }
}
}
}

```

7. Interface Gráfica (UsuariosUserControl)

A interface usa o **Windows Forms** com a biblioteca moderna Guna UI.

- Use um `DataGridView` para mostrar a listagem.
- Crie botões para: **Novo**, **Editar**, **Excluir** e **Atualizar**.

No evento `Load` do seu UserControl, você chama o banco de dados:

```

private async void UsuariosUserControl_Load(object sender, EventArgs e)
{
    if (DesignMode) return;
    _usuariosService = new UsuariosApiService();

    // Oculta os botões se não for Admin
    bool isAdmin = SessionManager.Instance.IsAdmin;
    btnNovo.Visible = isAdmin;
    btnEditar.Visible = isAdmin;

    await CarregarDadosAsync();
}

private async Task CarregarDadosAsync()
{
    var usuarios = await _usuariosService.GetAllAsync();

    gridUsuarios.Rows.Clear();
    foreach(var u in usuarios)
    {
        // Adiciona as colunas no GridView
        gridUsuarios.Rows.Add(u.Id, u.Nome, u.Email, u.Perfil);
    }
}
}

```

8. Validações

Sempre aplique regras de validação! Seja no frontend (para feedback rápido do usuário) ou no backend (por segurança).

- **Frontend (No clique de Salvar do Formulário Desktop):** `if (txtSenha.Text ≠ txtConfirmarSenha.Text) MessageBox.Show("As senhas não coincidem!");`
- **Backend (No Application Service):** também verifique senhas idênticas, obrigatoriedade de campos e evite emails duplicados com `await _userManager.FindByEmailAsync(dto.Email);`

05 DEBUGGING E RESOLUÇÃO DE PROBLEMAS

Se algo não funcionar, a melhor ferramenta é o **Breakpoint** (Bolinha vermelha no canto da linha):

- 1 Problemas na Tela:** coloque o breakpoint no botão Salvar do Desktop. Pressione `F10` (Step Over) para ler linha por linha. Verifique se as variáveis de texto estão capturando as digitações corretamente.
- 2 Problemas na API:** coloque o breakpoint no `UsuariosController`. Se a execução sequer chegar lá, significa que sua rota está errada ou que o token de autenticação falhou (você vai tomar um erro 401 Unauthorized ou 404 Not Found no desktop).
- 3 Problemas de Injeção de Dependência:** ocorreu um erro 500 no carregamento dizendo que a interface não pôde ser resolvida? Verifique se você não se esqueceu do `AddScoped` no `Program.cs`.

06

PARA PRATICAR

EXERCÍCIOS DE FIXAÇÃO

- Exercício 1:** modifique a API e o Desktop para que o campo "Email" seja listado junto do nome no DataGridView do form.
- Exercício 2:** crie um filtro na interface Desktop onde você pode digitar o nome e a tela vai esconder usuários que não contiverem a letra pesquisada (Pesquisa Local via LINQ).
- Exercício 3:** modifique a criação para não deixar cadastrar nomes em branco.

07

PARA PRATICAR

DESAFIOS EXTRAS

Para os avançados:

- 1 Alterar Senha do Usuário logado:** crie um botão "Meu Perfil" onde o usuário consegue alterar sua própria senha, mas precise inserir a "Senha Antiga" antes (usando `ChangePasswordAsync` do Identity).
- 2 Ativar / Inativar Usuário:** adicione um atributo booleano (verdadeiro/falso) no Banco de Dados para que o admin possa bloquear usuários temporariamente ao invés de apagá-los completamente (Exclusão Lógica).

08

PARA PRATICAR

CHECKLIST DE SUCESSO

Marque cada item quando finalizar:

- ☐ DTOs criados e configurados.
- ☐ Interface `IUsuariosService` estruturada.
- ☐ Classe `UsuariosService` implementada (CRUD) usando ASP.NET Identity.
- ☐ Interface injetada no container (Program.cs).
- ☐ `UsuariosController` construído e protegido.
- ☐ API testada via Swagger com sucesso.
- ☐ `UsuariosApiService` no Desktop enviando requisições REST corretamente.
- ☐ Janela de formulário `UsuarioFormDialog` responsiva e salvando dados.
- ☐ Grid de visualização (`UsuariosUserControl`) renderizando os usuários no painel principal.



Fim do Manual

Boa programação!